Module 4: Evaluation

Objects & Methods

Guest starring Prototypal Inheritance, yayyyyyy…

1.

The Four* Pillars of OOP

**or at least the ones we care about right now.*

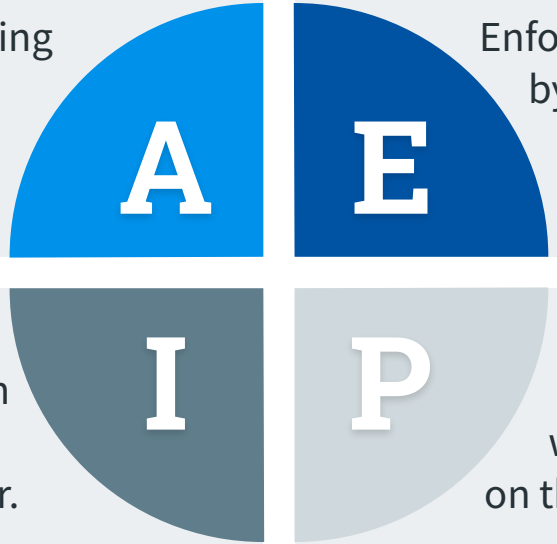
The Four Pillars of OOP

ABSTRACTION

Manage complexity by defining an API (interface) to simplify usage for users by omitting unimportant details.

ENCAPSULATION

Enforce a separation of concerns by hiding the implementation state from the users in favor of meaningful operations.



Allow building off of a parent class as a mechanism for code reuse, sharing, and extension with new behavior.

INHERITANCE

Allow objects to be used generically in algorithms, where the behavior depends on the runtime implementation.

POLYMORPHISM

Polymorphism

“Multiple Forms”

Polymorphism allows code (methods) to be used generically with multiple objects, each with their own definition.

- Methods use dynamic dispatch (i.e. runtime types) for the receiver.
  - Aka, the object’s class.
- Polymorphism is a key part of most modern languages / software
  - Enables many abstractions

Styles of Polymorphism

Languages have varying approaches for polymorphism (or even multiple!).

- Subtype Polymorphism
  - Nominal subtyping, e.g. class inheritance or interfaces/traits
- Ad-hoc Polymorphism
  - Structural subtyping, e.g. duck typing & dynamic/runtime behavior
- Parametric Polymorphism (Generics)
  - Polymorphism over types, e.g. higher order functions.

Inheritance

Behavior Sharing/Extension

Inheritance is a feature for code reuse, sharing, and behavior extension.

- Classes can “inherit” behavior (and even state) from other classes.
  - Classes can override methods to further customize behavior.
- Interfaces give subtyping & poly-morphism, inheritance adds reuse.
- Multiple entities control behavior!

Inheritance is *Powerful*

Inheritance is one of the most powerful language features widely used in OOP.

- Allows sharing code/state, increasing the coupling between classes.
- Methods, even private ones, are written generically (via polymorphism).
- Designing for inheritance requires knowing what subtypes (might) do.
- Consequently, also highly misused!
  - Enterprise Java & friends :(

```javascript
const dog = {
  makeSound() {
    console.log("Woof!");
  }
}
dog.makeSound(); //Woof!

function doAnimalThings(animal) {
  animal.makeSound();
}
doAnimalThings(dog); //Woof!
```

```javascript
class Animal {
  makeSound() {
    console.log("Animal?");
  }
}
class Dog extends Animal { … }

const dog = new Dog();
dog.makeSound(); //Animal?

function doAnimalThings(animal) {
  animal.makeSound();
}
doAnimalThings(dog); //Animal?
```

Polymorphism vs Inheritance

2.

Prototypes

You make prototype objects, and then … make new instances. We don't need classes to make lots of similar objects… Objects inherit from objects. What could be more object oriented than that? ~Douglas Crockford

Prototypes

Definition

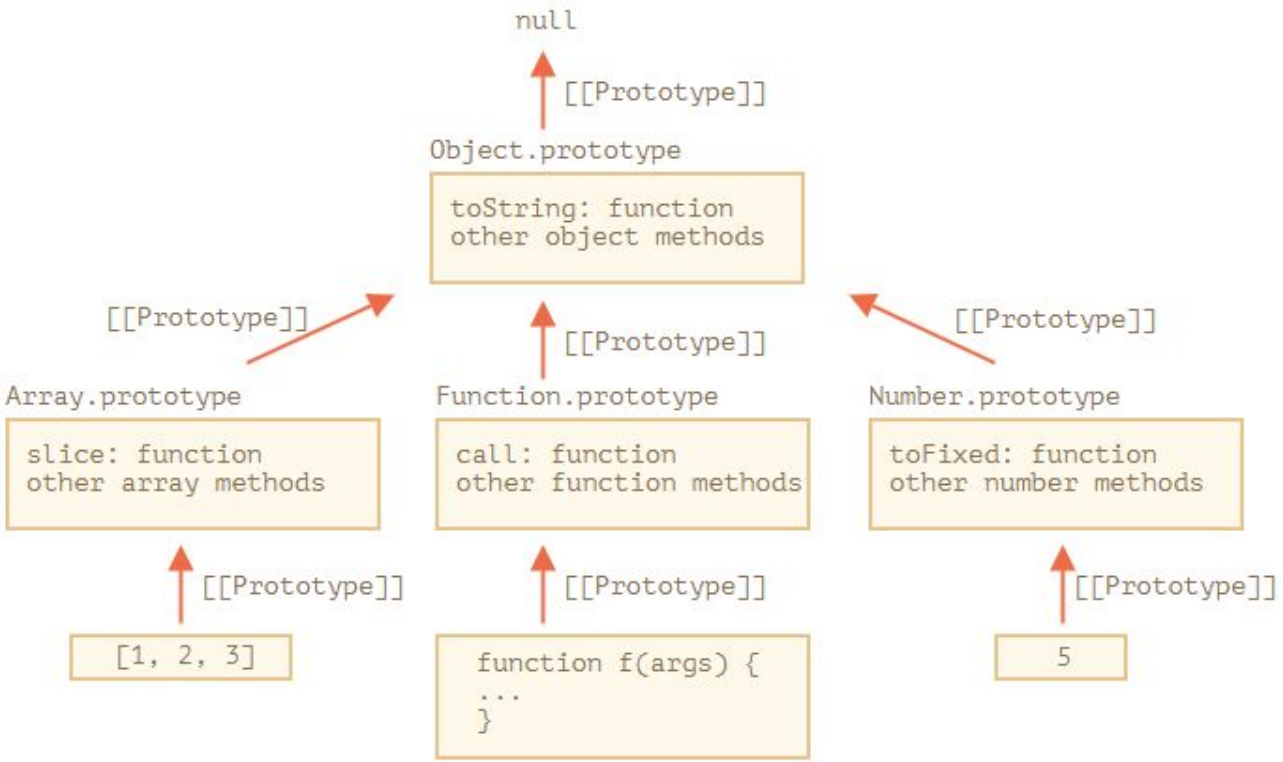
The **prototype** is an object that defines inherited fields/methods.

- An object's prototype is effectively it's parent class, just as an object.
  - Inheritance = prototype
- Both the prototype field and object are (typically) mutable.
  - Can change inheritance, define new methods, etc.

Prototype Chain

The prototype chain is the lookup of prototypes for each object (until null).

- Integer -> Number -> Object
  - Effectively, it's the class hierarchy!
- Method lookups are performed by navigating through the prototype chain.
  - If an object doesn't define it's method, call it on the prototype.
  - If the prototype is null (end of chain) the method is thus undefined.



JavaScript Prototype Chain

Module 4: Evaluation

Function Scoping

Conjunction junction, what's your function?

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals